

SİBER GÜVENLİK LABORATUVARI

KRİPTOGRAFİ DENEY RAPORU

Onur Alkan | 232923044 | 06.10.2026

Amaç

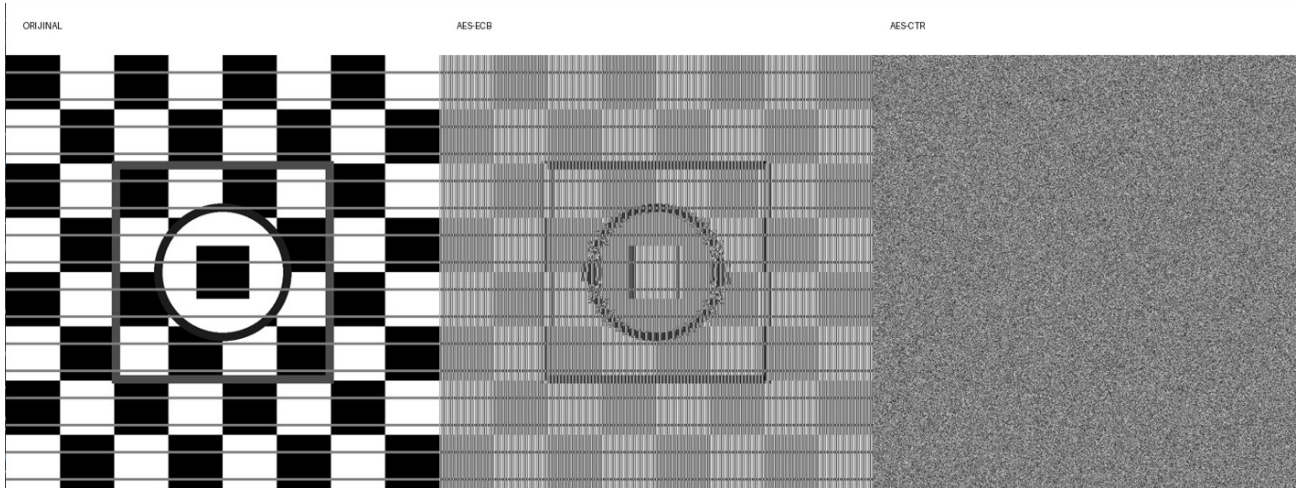
Bu çalışmada blok şifreleme modlarının görsel veri üzerindeki etkisini, CTR modunda bütünlük korumasının olmamasını, AES-GCM ile kimlik doğrulamalı şifrelemenin sağladığı farkı ve SHA-256 ile Argon2id arasındaki çalışma süresi farkını kendi bilgisayarım üzerinde gözlemledim.

Çalışma Ortamı

Bileşen	Kullanılan sürüm / değer
Python	3.14.4
cryptography	50.0.2
argon2-cffi	25.1.0
Pillow	12.3.0
AES anahtar uzunluğu	128 bit

1. Deney - AES-ECB ve AES-CTR Görsel Karşılaştırması

Yöntem: Tekrar eden desenleri belirgin olan 1024x1024 piksellik bir test görüntüsünü gri tona dönüştürdüm. Aynı AES-128 anahtarı ile görüntü verisini önce ECB, sonra CTR modunda şifreledim ve sonuçları aynı ölçekte yan yana karşılaştırdım.



Şekil 1. Orijinal görüntü, AES-ECB ve AES-CTR çıktılarının karşılaştırılması

Gözlem ve değerlendirme: ECB çıktısında orijinal görüntüdeki büyük geometrik şekiller ve tekrar eden yapı hâlâ seçilebiliyordu. CTR çıktısı ise rastgele gürültü görünümündeydi ve görüntünün yapısı seçilemiyordu. Bunun nedeni ECB modunda aynı düz metin bloklarının aynı şifreli bloklara dönüşmesidir. CTR modunda ise sayaç tabanlı anahtar akışı her blokta farklılaştığı için tekrar eden desenler gizlenir.

2. Deney - CTR Bütünlük Problemi ve AES-GCM Karşılaştırması

Yöntem: Başlangıç mesajı olarak "Tutar: 100 TL" kullandım. CTR ile elde edilen şifreli veri üzerinde, anahtarı kullanmadan yalnızca 100 -> 900 dönüşümüne karşılık gelen XOR farkını uyguladım ve çözüm sonucunu "Tutar: 900 TL" olarak elde ettim. Ardından CTR deneyindeki aynı 100 -> 900 XOR farkını AES-GCM şifreli veri bölümüne uyguladım; doğrulama etiketini değiştirmeden bıraktım.

Kontrol	Sonuç
CTR - orijinal mesaj	Tutar: 100 TL
CTR - hedef mesaj	Tutar: 900 TL
CTR şifreli veri	1212f6f5aa7a3e0246ab95c2f7
CTR değiştirilmiş veri	1212f6f5aa7a3e0a46ab95c2f7
CTR çözüm sonucu	Tutar: 900 TL
AES-GCM aynı 100 -> 900 değişikliği	REDDDEDİLDİ (InvalidTag)

Gözlem ve değerlendirme: CTR tarafında yalnızca ilgili şifreli byte değişmesine rağmen çözülmüş düz metin 100 TL yerine 900 TL oldu. Bu, CTR modunun tek başına gizlilik sağladığını fakat bütünlük doğrulaması sağlamadığını gösterdi. AES-GCM tarafında ise CTR'de başarılı olan aynı 100 -> 900 değişikliği doğrulama etiketi nedeniyle InvalidTag hatasıyla reddedildi. Böylece AES-GCM'nin şifreli verideki oynama girişimini tespit ettiği doğrudan gözlemlendi.

3. Deney - SHA-256 ve Argon2id Süre Karşılaştırması

Yöntem: SHA-256 çok hızlı olduğu için 100.000 tekrar çalıştırıp işlem başına ortalama süreyi hesapladım. Argon2id için beş tekrar yaptım ve time_cost=3, memory_cost=65536 KiB (64 MiB), parallelism=2 parametrelerini kullandım.

Ölçüm	SHA-256	Argon2id
Tekrar sayısı	100.000	5
Toplam / ortalama süre	0,071847 s toplam	95,450 ms ortalama
İşlem başına ortalama	0,000718 ms	-
En kısa süre	-	92,433 ms
En uzun süre	-	97,784 ms
Yaklaşık süre oranı	-	132.851 kat daha uzun

Gözlem ve değerlendirme: SHA-256 genel amaçlı bir özet fonksiyonu olduğu için tek işlem çok kısa sürdü. Argon2id ise parola saklama amacıyla zaman ve bellek maliyeti oluşturacak şekilde tasarlandığından belirgin biçimde daha yavaştı. Yaklaşık 132.851 katlık oran yalnızca bu bilgisayarda, bu parametrelerle yapılan ölçümü ifade eder; evrensel bir sabit değildir. Bu kasıtlı maliyet, yüksek hacimli parola tahminlerini daha pahalı hâle getirir.

Genel Sonuç

Deneyler sonunda üç temel sonucu doğrudan gözlemledim: ECB modu şifreleme yapmasına rağmen tekrar eden veri desenlerini sızdırabilir; CTR gizlilik sağlasa da tek başına bütünlük koruması sağlamaz; AES-GCM ise şifreleme ile birlikte veri bütünlüğünü de doğrular. Ayrıca parola saklama için tasarlanan Argon2id'in, genel amaçlı SHA-256'ya göre bilinçli biçimde çok daha maliyetli çalıştığını ölçtüm.

Not: CBC modundaki padding hata mesajı konusu ders slaytında öğretim elemanı gösterimi olarak belirtildiği için ayrı bir öğrenci deneyi olarak raporlanmadı.